The Game of Life

Prepared by: Rene Andre Bedonia Jocsing

Published: October 13, 2025 **Deadline:** November 27, 2025

This laboratory assignment implements Conway's Game of Life using C and Assembly. You will create an idle simulation game that handles multi-dimensional arrays, buffer management, and efficient pattern detection.

Background

The Game of Life is a cellular automaton devised by the British mathematician John Horton Conway in 1970. It is the best-known example of a cellular automaton that demonstrates how complex patterns can emerge from simple rules.

Game Rules

The universe consists of a 2D grid of cells, each either alive (1) or dead (0). A generation follows these rules:

1. Any live cell with 2 or 3 live neighbors survives
2. Any dead cell with exactly 3 live neighbors becomes alive
3. All other live cells die, and all other dead cells stay dead

Learning Objectives

By the end of this laboratory exercise, students should be able to:

- Implement 2D array traversal using nested loops in assembly
- Apply conditional logic for cellular automaton rule implementation
- Use string instructions (`rep movsd`, `repe scasd`, `repe cmpsb`) for efficient buffer operations
- Interface assembly and C using proper calling conventions
- Implement pattern detection algorithms for stability and homogeneity checking
- Manage memory allocation for dynamic grid storage

Task

Create a program that simulates Conway's Game of Life:

1. Randomize an initial 10 x 10 grid (seeded zero'th generation)
2. For the initial grid, **all border nodes are set to be dead**
3. Display the current generation in a readable format
4. Calculate the next generation based on the rules of the game
5. Run simulation until either grid is stable or homogenous
6. Show statistics by the end of the simulation

Implementation Details

- Use a 2D array to represent the cell grid
- Implement neighbor counting using nested loops
- Display each generation with clear formatting
- Track generation number and live cell count
- At the bare minimum, use two buffers to accomplish the task
- Observe good C coding and documentation practices

For each cell at position (row, col):

- Count live neighbors using some algorithm of choice
- Apply rules:
  - Dead cell with exactly 3 neighbors → becomes alive
  - Live cell with 2 or 3 neighbors → survives
  - All other cases → cell dies or stays dead

You are allowed to **interface C and Assembly**. Provided, however, that the **number of lines in your C source code does not exceed the number of lines in your Assembly source code**. This means that how the program is structured is left to your discretion.

Expected Output

See this recording for an example simulation.

Testing

Verify your implementation handles:

- Correct application of the rules of Conway's Game of Life
- Proper buffer management without data loss or corruption
- Accurate homogeneity or stability detection

Laboratory Defense

The instructor will be available during scheduled laboratory hours in the assigned room for defense or consultation. You will be catered to at a first-come first-served basis. It is mandatory to present your code, answer inquiries, and perform live programming if the instructor deems it necessary to further check your understanding.

If a laboratory defense for an activity fails to be conducted within **one month** of its assignment, the instructor will be forced to give a failing grade. Scoring will be done during the laboratory defense (no defense, no grade policy). Extensions may be granted due to extraordinary circumstances, but ultimately remains up to the judgment of the instructor.

Academic Honesty

The usage of Large Language Models (e.g. ChatGPT, Claude, Deepseek, etc.) to generate code is considered cheating. As cheating is against the university's code of ethics, it is subject to failure in the course and harsh disciplinary action.

Rubric for Programming Exercises (50 pts)

Criteria (10 Points Each)	Excellent (9-10)	Good (6-8)	Fair (3-5)	Poor (0-2)
Program Correctness	Program executes correctly with no syntax or runtime errors, meets/exceeds specifications, and displays correct output	Program executes and outputs with minor errors, yet meets specifications	Program executes and outputs with major errors, yet somehow meets specifications	Program does not execute or does not meet specs
Logical Design	Program is logically well-designed with excellent structure and flow	Program has slight logic errors that do not significantly affect the results	Program has significant logic errors affecting functionality	Program logic is fundamentally incorrect
Code Mastery	Programmer demonstrates excellent mastery over the program's code	Programmer demonstrates adequate mastery over the program's code	Programmer demonstrates fair mastery over the program's code	Programmer demonstrates poor mastery over the program's code
Engineering Standards	Program is stylistically well designed from an engineering standpoint	Slight inappropriate design choices (i.e., poor variable names, improper indentation)	Severe inappropriate design choices (i.e., code repetition, redundancy)	Program is poorly written
Documentation*	Program is well-documented: comments exist for clarity, not redundancy	Missing one required comment or some redundant comments	Missing two or more required comments or many redundant comments	Most documentation missing or most documentation is redundant

***Remember: "Code tells you how, comments tell you why."** — Jeff Atwood, co-founder of Stack Overflow and Discourse

---

See the laboratory manual for submission requirements.